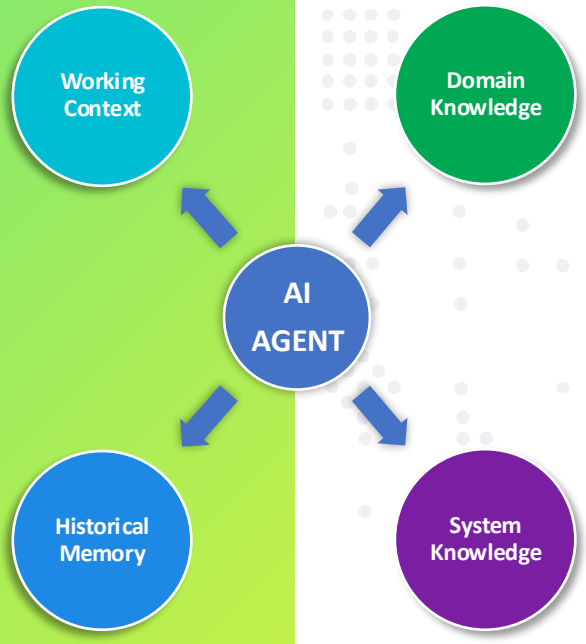
STAREAST · 2026

RYRUS

Context Engineering for AI-Driven QA

Memory, Agents, and the Future of Test Generation



Testing agents become dramatically more capable when they operate with structured memory and context.

IMAGINE THIS

Your competitor just shipped in 2 weeks what took you 6 months.

Same AI model. Same team size. Same budget.
But their AI agent writes tests that **actually work**.
Yours hallucinates 40% of the time.

The difference?
Context Engineering.
Not better prompts.
Not a bigger model.

Time to Market
6+ months → **2 weeks**

Test Accuracy
60% usable → **95% usable**

Rework Cost
$500K+ → **Near zero**

The teams winning with AI aren't using secret models.
They're **engineering context** — and it's a skill you can learn today.

Code without context is just syntax.

Every bug you've ever fixed was a context problem in disguise.

What the code says		What you need to know
`user. isActive = false;`	→	Deactivation triggers 47 downstream events, invalidates 12 caches, and sends 3 notifications
`setTimeout(fn, 100);`	→	This delay exists because the payment gateway needs 80ms to release the lock on mobile
`if (count > 1000) return;`	→	Added after the 2019 outage when a customer uploaded 50K items and crashed prod

70%
of dev time is reading code

40%
of that is understanding WHY

0%
is captured in the code itself

Tests don't fail because of bad code. They fail because of missing context.

What your test checks		What it actually needs to know
`expect(response.status).toBe(200);`	→	API returns 200 even on partial failures; check response.data.errors array for actual status
`await page.click('#submit');`	→	Button is disabled for 300ms after form validation; add waitForEnabled() or test flakes 15% of the time
`assert user.balance == 0`	→	Balance rounds to 2 decimals in DB but 4 in memory; use approx match or test fails on $0.001 diff

80%
of flaky tests are context issues

3x
longer to debug without context

60%
of test maintenance is re-learning intent

Your test suite is only as smart as the context it has access to.

Most AI testing agents operate Statelessly

Re-Reasoning Every Step

Every test step requires the agent to reason from scratch about the same application

Rediscovering UI Patterns

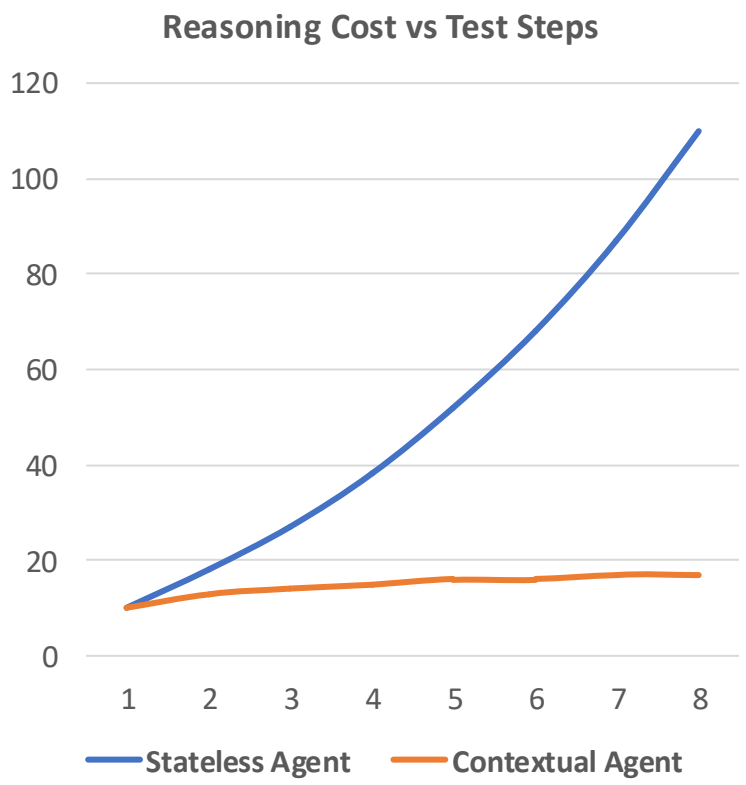
Agents re-discover identical elements, locators, and navigation flows repeatedly

Duplicate Test Creation

Without memory, similar tests are generated over and over with no awareness of existing coverage

Context Lost Between Runs

Knowledge about application behavior, failures, and fixes vanishes after each session



Without memory, AI constantly recomputes the world.

The systematic design of what an AI knows during a task

"Context Engineering is the systematic design and delivery of information, tools, and memory that an AI agent requires to perform tasks reliably and efficiently."

1 Working Context

IMMEDIATE TASK STATE

Current DOM state, active steps, in-progress test actions, and immediate agent goals.

2 Historical Memory

WHAT HAPPENED BEFORE

Past test runs, previously discovered locators, known page structures and navigation paths.

3 System Knowledge

HOW THE APP IS BUILT

Application architecture, component libraries, page flow maps, and navigation topology.

4 Domain Knowledge

BUSINESS CONTEXT

Business rules, user workflows, valid test data, SAP transactions, and organizational logic.

Context engineering is the new prompt engineering.

Testing demands layered, cumulative knowledge

Existing Test Coverage
Know what's already been tested so you don't duplicate work

Application Structure
Understand UI hierarchies, routing, component libraries

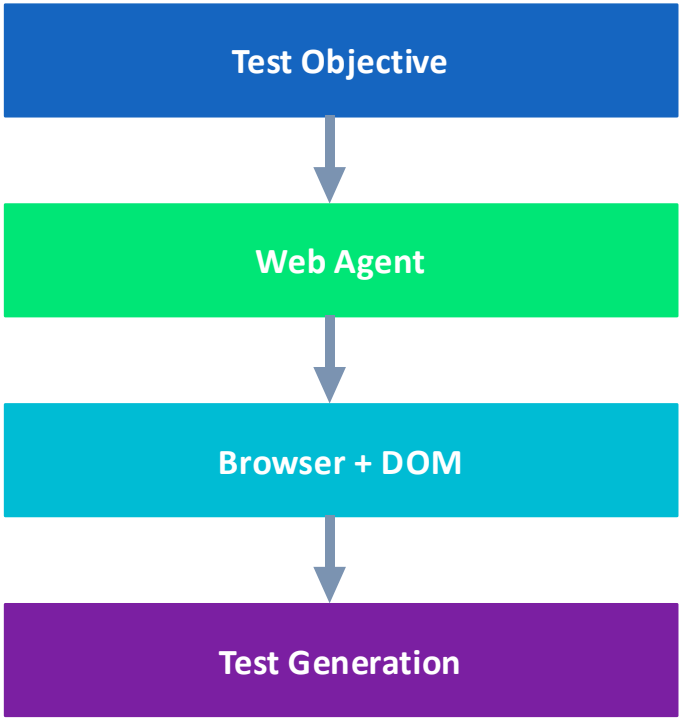
Historical Failures
Learn from bugs and regressions to target high-risk areas

Business Workflows
Map business processes to meaningful, end-to-end test scenarios

WITHOUT CONTEXT	WITH CONTEXT
✗ Agents re-explore the same screens	✓ Builds incremental intelligence
✗ Duplicate tests accumulate silently	✓ Extends known test coverage
✗ No awareness of prior coverage	✓ Learns from previous sessions
✗ Every run costs full compute	✓ Compound efficiency gains

AI builds incremental intelligence with context.

Converting user objectives into executable tests



UI State Reader

Continuously observes DOM, identifies interactive elements, maps component hierarchy and current page state

Navigation Generator

Plans intelligent navigation sequences across pages, handling SPAs, dynamic content, and multi-step workflows

Test Case Builder

Transforms captured user flows into structured, reusable test steps with precise selectors and assertions

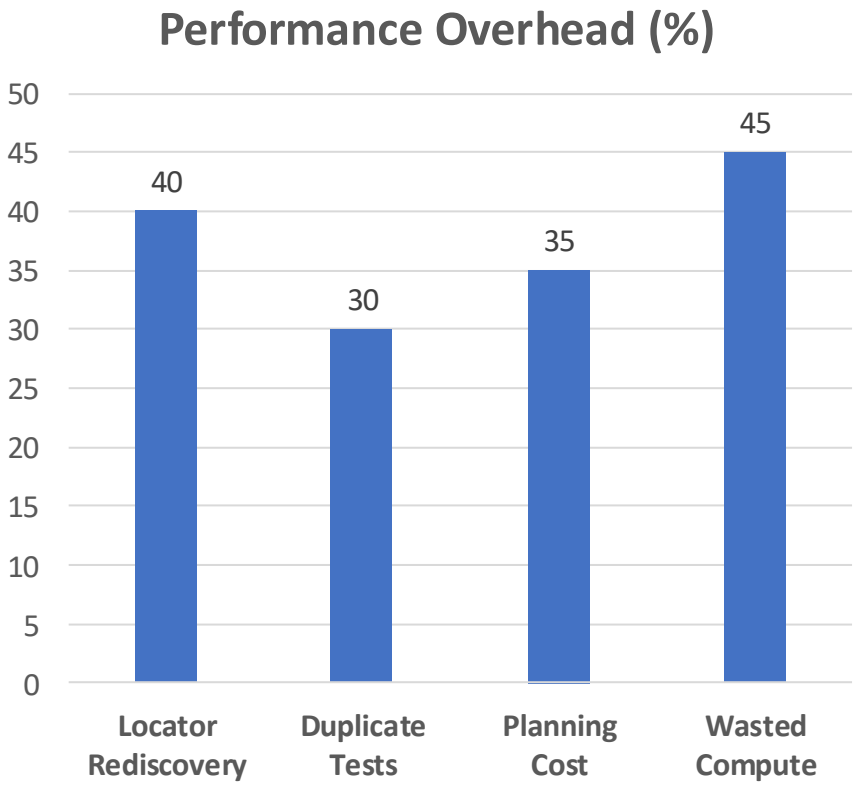
Agent objective: Convert user goals into executable tests. The agent sits directly on top of the browser and builds tests based on objectives.

Observed behavior before memory integration

14×

the same login button discovered across generated tests

Repeated Element Discovery	Duplicate Locator Generation	Redundant Page State Reasoning
HIGH	HIGH	HIGH



Without memory, AI constantly reinvents the wheel — locators, patterns, and page maps discovered repeatedly.

A persistent knowledge layer for the testing agent

EXTENDED MEMORY LAYER — Retaining knowledge between sessions and across tests

Known Locators

CSS selectors, XPaths, and unique identifiers for every discovered UI element — reused instead of re-discovered

Existing Test Steps

Complete test step sequences from previous test generation sessions, available for reuse and extension

Page Structures

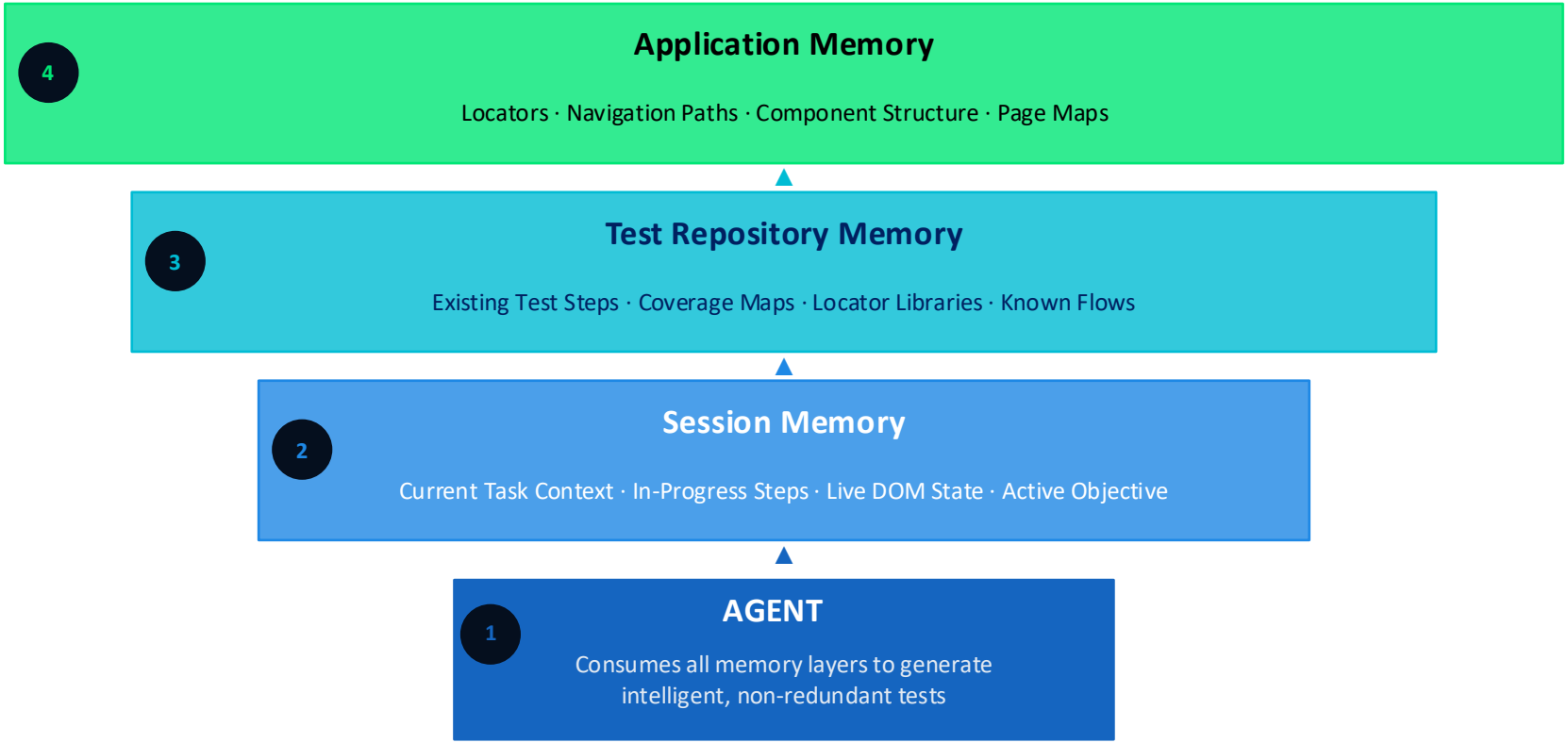
Full DOM structure snapshots, component maps, navigation flows and application topology knowledge

Exploration Paths

Previously successful navigation sequences, dead-end detection, and optimal routing between UI states

Adding extended memory allows the web testing agent to build tests faster and avoid redundant reasoning.

Three layered stores powering the contextual agent



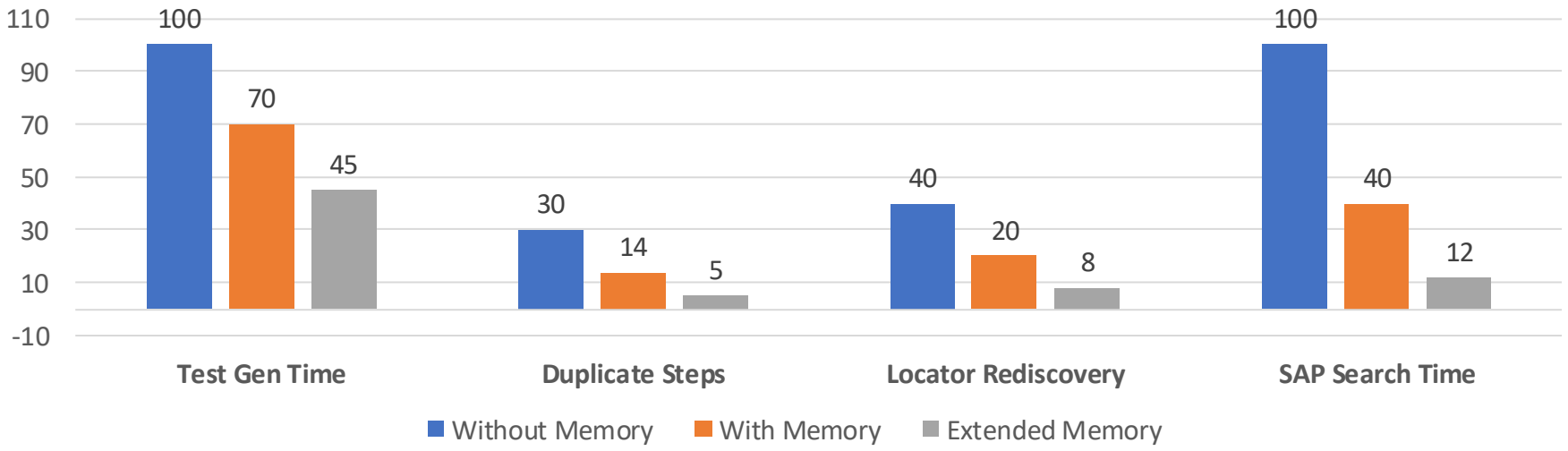
Layered memory architecture enables the agent to access the right context at every level of test generation.

Real performance data after memory integration

55%
Faster test generation

83%
Fewer duplicate steps

80%
Less locator rediscovery



Memory converts exploration into structured learning — each run makes the next run faster.

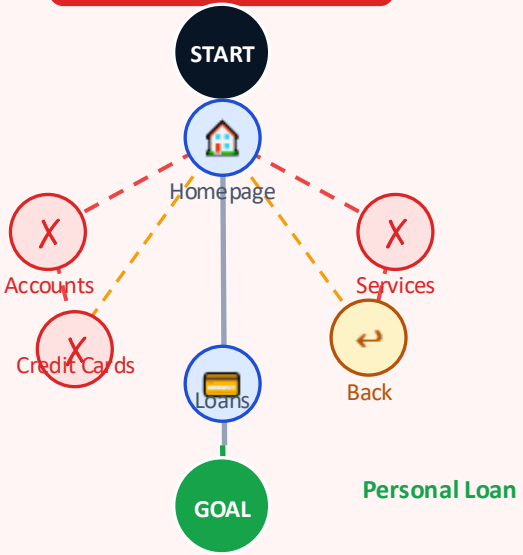
Our solution to context engineering/agentic/autonomous testing is SEER

What is SEER?

How do we enable **context differently** than **what is possible?**

Memory In Action: Personal Loan Scenario

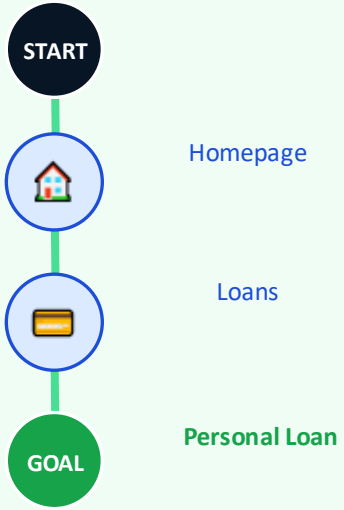
✗ Without Memory



7 steps • ~850 tokens • high cost

vs

✓ With Memory

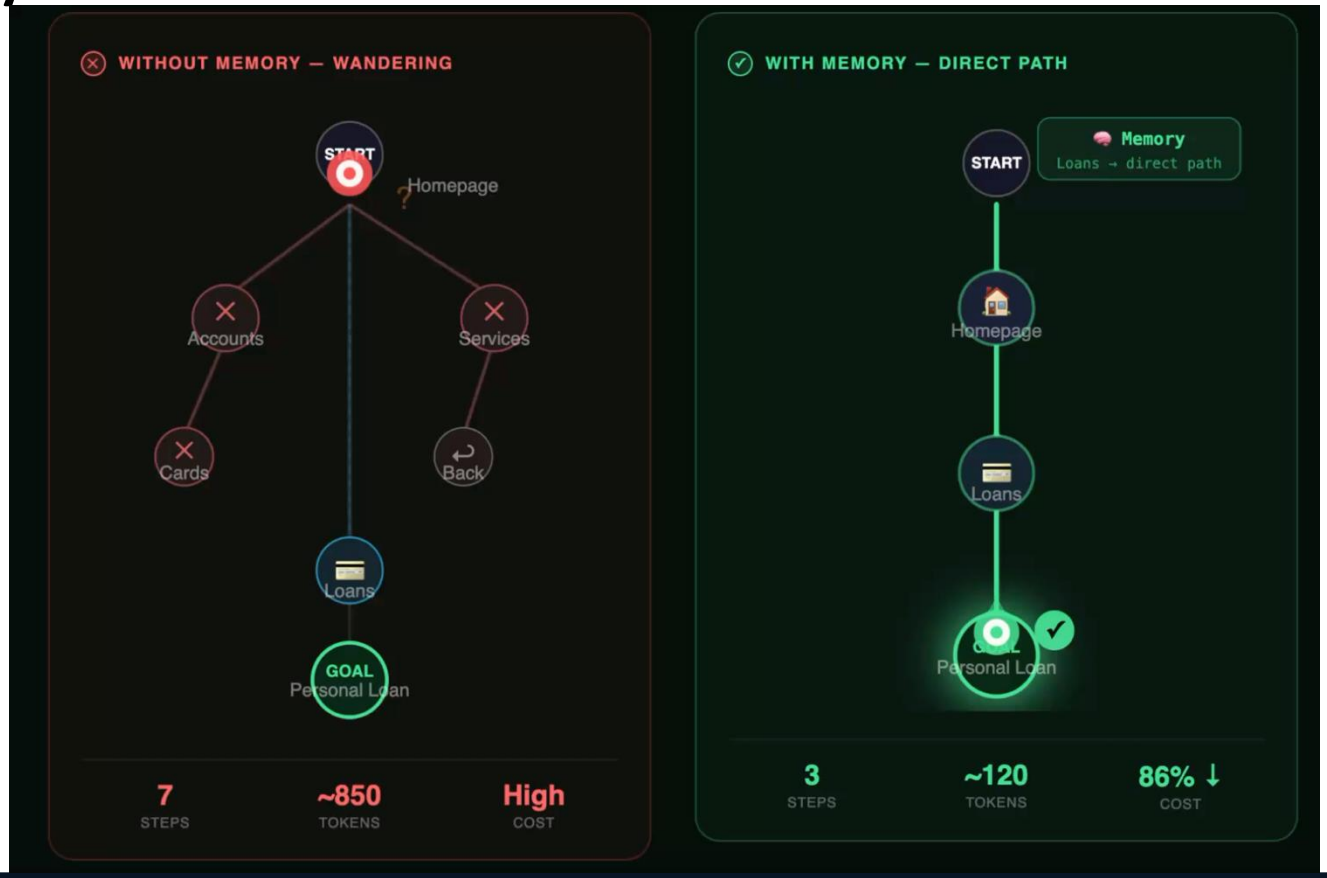


3 steps • ~120 tokens • 86% less cost

🧠 Memory: "Loans > Personal Loan is the direct path"

86% fewer tokens • Less inference = less cost • Memory pays for itself

Memory In Action:



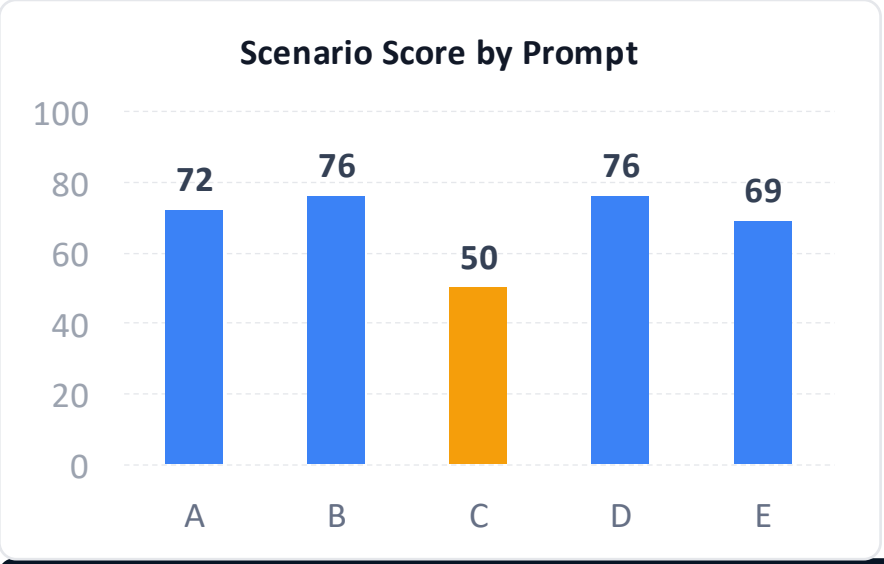
86% fewer tokens • Less inference = less cost • Memory pays for itself

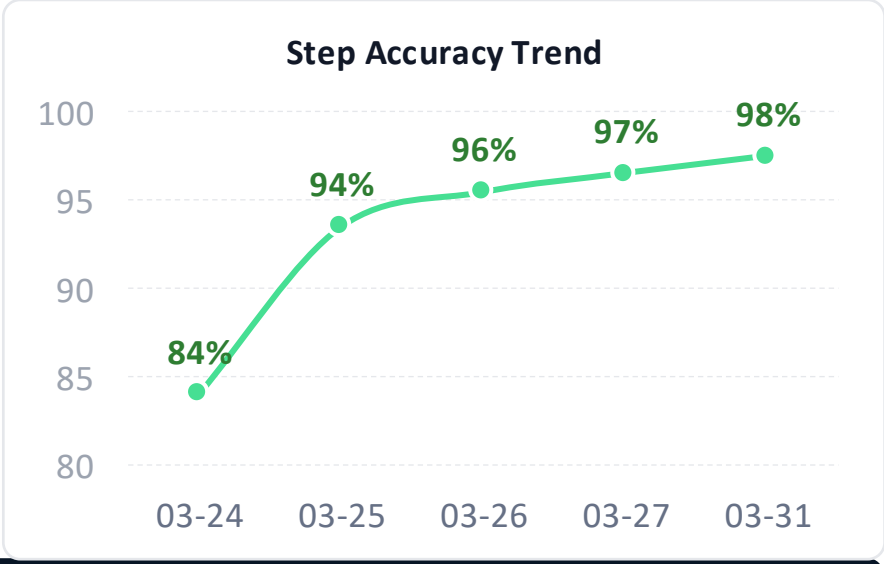
Qyrus SEER Evaluation Dashboard

Total Scenarios
25
5 Prompts Tested

Avg Scenario Score
70.5%
Overall Quality

Avg Step Accuracy
95.1%
Assertion Index: 0.15





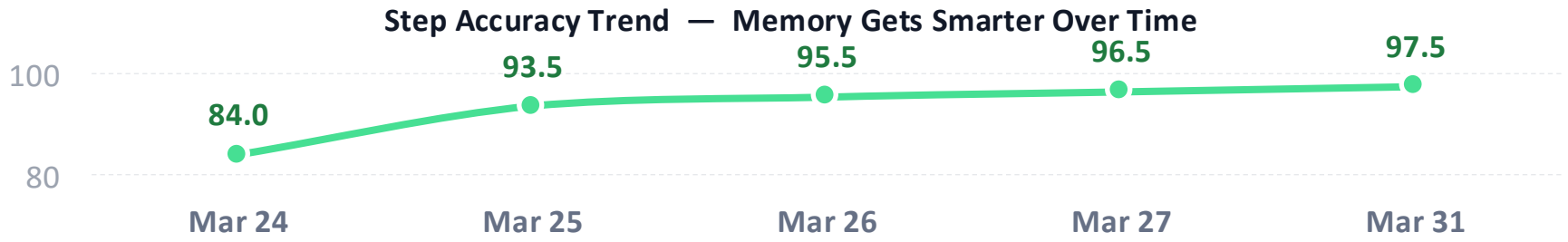
Takeaway: 95.1% step accuracy — improves from 84% to 97% as memory accumulates

The Memory Effect

25
Scenarios
across 5 prompt strategies

70.5%
Avg Quality Score
scenario-level accuracy

95.1%
Step Accuracy
assertion index: 0.15



Context + Memory = 95% accuracy • 84%→97% in 7 days • it only gets better

The agent understands what already exists

MEMORY SOURCES

Test Repository

Complete library of previously generated and manually authored test cases

Locator Libraries

Indexed catalog of element selectors organized by page, component, and function

UI Components

Known component behaviors, state transitions, and interaction patterns

WHAT THE AGENT NOW UNDERSTANDS

Which flows already exist?

Queries test repository to avoid re-creating covered scenarios

Which screens are covered?

Maps coverage by page, route, and component to identify gaps

Which elements are known?

Checks locator library before running new discovery on the DOM

The agent maintains context about existing tests and locators to generate better, non-redundant tests.

Stateless testing cannot scale to enterprise complexity

100s
of SAP Transactions

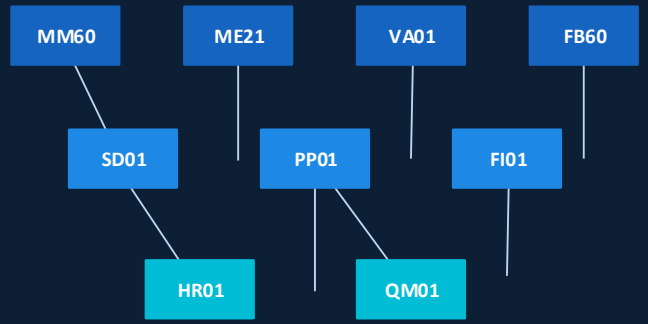
1000s
of Form Fields

∞
Integration Paths

WHY STATELESS TESTING FAILS HERE

- ✗ Cannot map hundreds of transactions without prior knowledge
- ✗ Every test session re-explores the same massive UI surface
- ✗ No awareness of which transactions feed into which workflows
- ✗ Test data discovery requires understanding business configurations

SAP Landscape



Enterprise systems like SAP have massive surface area that stateless agents cannot navigate effectively.

Agents that understand your SAP landscape

AGENT CONTEXT INCLUDES

Customer Configuration

Org-specific SAP customizations, chart of accounts, company codes, and module activations

SAP Workflows

Standard and custom workflow definitions, approval chains, and transaction dependencies

Data Relationships

Material master, vendor/customer master, and cross-module data linkages

CAPABILITIES UNLOCKED

Find Correct Test Data

Locates valid material numbers, vendor IDs, and company-specific data automatically

Build Meaningful Tests

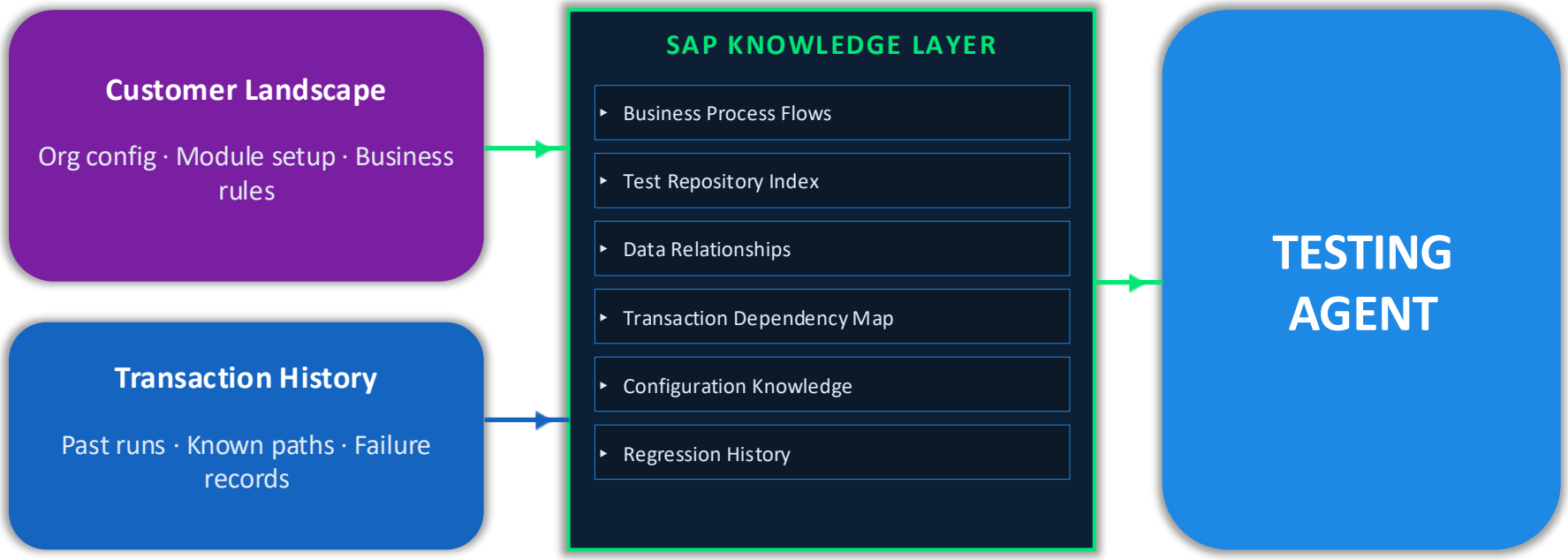
Generates end-to-end scenarios that match actual business processes, not just UI interactions

Analyze Change Impact

Identifies which tests are affected by configuration or code changes before they run

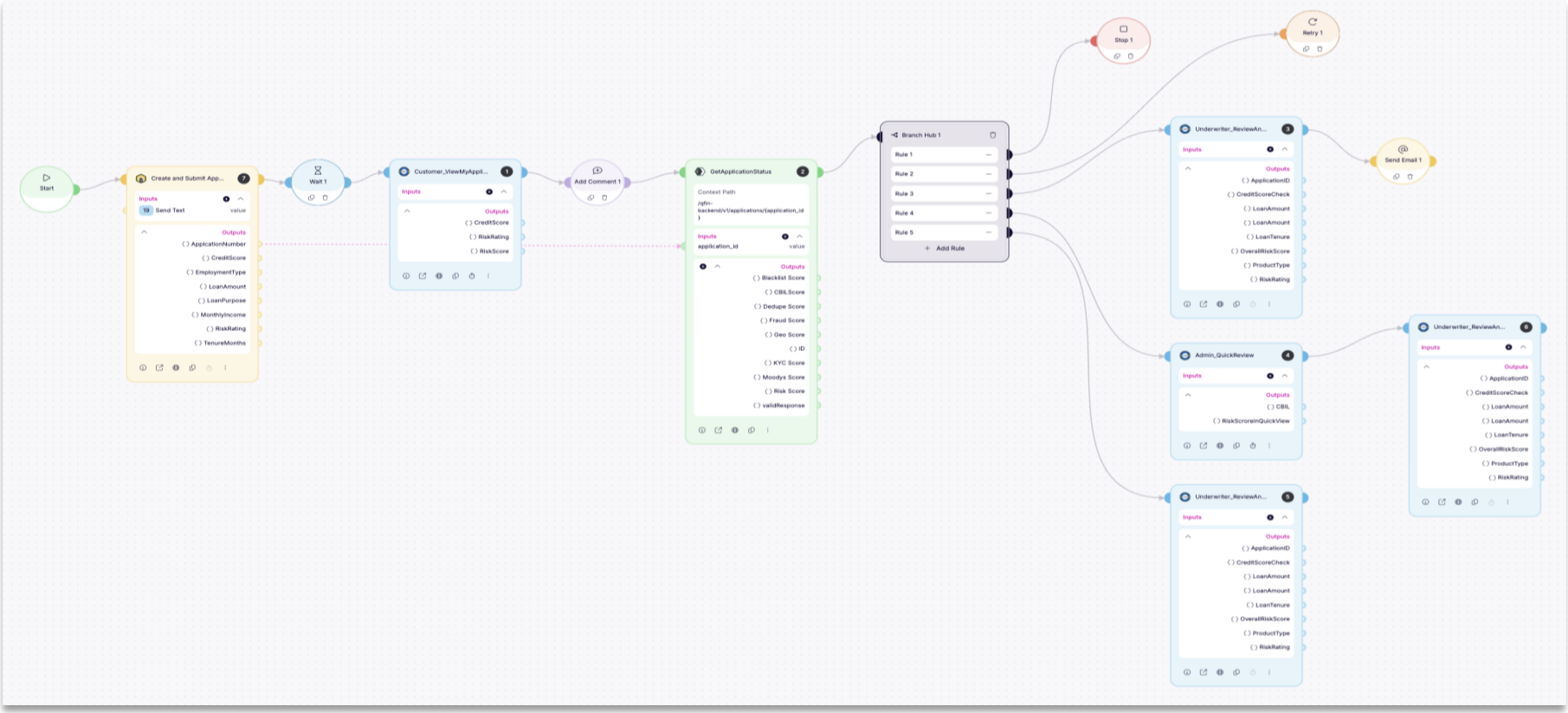
Context awareness is especially important for enterprise systems like SAP where agents must understand customer landscapes.

Accumulated knowledge drives intelligent enterprise testing



Agents accumulate knowledge of the customer landscape to perform testing more effectively with each iteration.

Cross Platform Test Orchestration



AI-driven change analysis replaces manual test selection

TRADITIONAL APPROACH

1. Code Change
2. Manual Review
3. Guess Impact
4. Run All Tests
5. Hope Nothing Broke

VS

CONTEXT-AWARE APPROACH

1. Code Change Detected
2. Dependency Graph Query
3. AI Impact Analysis
4. Targeted Test Selection
5. Precise Regression Run

AI identifies impacted transactions, workflows, and tests — eliminating the guesswork from regression cycles.

Four phases of testing evolution

01	02	03	04
Manual Testing	**Automation Frameworks**	**AI Test Generation**	**Context-Driven AI**
Human testers execute test cases by hand. High cost, low scale, high variability.	Selenium, Cypress, and record-and-playback tools codify repetitive regression flows.	AI generates test cases from objectives, exploring UI automatically without manual scripting.	Agents with memory and domain knowledge deliver compounding, intelligent test coverage.

The competitive advantage in AI testing will come from context engineering — not just from using AI to generate tests.

Context-driven AI testing is the next frontier — and it's happening now.

Building AI testing systems that scale with context

01 Treat Memory as Infrastructure

Memory is not a feature — it's the foundational layer your testing agents depend on. Invest in it like you invest in test infrastructure.

02 Capture Knowledge Continuously

Every test run should write back to memory. Passive knowledge accumulation is what separates a contextual agent from a stateless one.

03 Connect Agents to Test Repos

Agents that cannot query existing test libraries will always produce duplicate coverage. Repository integration is non-negotiable.

04 Preserve Application Knowledge

Locators, navigation paths, and component maps should survive between sessions. Rebuild nothing that can be remembered.

Without these principles, AI testing systems plateau quickly and fail to deliver compounding value.

Immediate actions to build context-engineering capability

1 **Build Structured Test Repositories**

Invest in organizing existing tests with rich metadata — screen coverage, flow type, business module, and component tags. This becomes the foundation for agent memory.

Immediate

2 **Capture Locator Knowledge**

Implement systematic locator indexing so every discovered element is stored and reusable across test sessions. Eliminate locator re-discovery permanently.

Q1 Priority

3 **Preserve Application Workflows**

Document and store end-to-end workflow maps for your critical business processes. Give your agents the domain knowledge they need to build meaningful tests.

Q1-Q2

4 **Introduce Contextual Agents**

Pilot AI testing agents with memory integration on a single critical module. Measure test generation speed and duplication rates before and after.

Q2 Pilot

Testing organizations must treat context engineering as a core capability — not an experiment.

Prompt **engineering** made **AI**

usable.

Context **engineering** will make **AI**

reliable.

*In **testing**, **reliability** is **everything.***

"The Future of QA is Context-Aware Agents" · StarEast 2026 · Qyrus

How Qyrus Helps

People + Technology — the two things you need to make AI work in testing.

Baked-In Context Engineering

Every test run, every element touched, every page visited — all in persistent memory baked into our workflows.

Memory Architecture & Pipelines

Purpose-built memory architecture and pipelines to power context-aware AI testing at scale.

Expert Team

A dedicated expert team that helps others achieve high efficiencies implementing AI in testing.

People + Technology = Context-Aware Testing

The current state

You're Using
AI for Testing.

BUT

THE QUESTION NO ONE IS ASKING

Who's Testing *your AI?*

At its core — AI runs on APIs. And API quality is your responsibility.

You're using AI for testing. But who's *testing your AI?*

87%
of teams use AI-assisted tools in dev

23%
have a mature AI testing strategy

4.2×
more API failures in AI-heavy systems

QAPI Agent · VS Code Extension · Free

Request builder · AI-generated assertions
Agentic chat · Import cURL, OpenAPI, Postman
CodeLens · Express, Flask, FastAPI, C#

LLM Evaluator · Python SDK · pip install qyrusai

RAG eval · Faithfulness + relevance scoring
MCP eval · Tool selection + arg validation
Batch async · pip install Qyrusai

Qyrus: *Test APIs. Evaluate AI. Ship with confidence.*

qAPI Agent + LLM Evaluator

Two Qyrus tools to test APIs and evaluate AI — free to use

QAPI Agent · VS Code · Free

API

Interactive API Testing

Full request builder — headers, auth, body. Syntax-highlighted responses.

AI

AI Assertion Generation

Auto-generate smart test assertions from any response. Review inline.

Chat

Agentic Chat

Ask the agent to write tests, mocks, or audits using your code as context.

Imp

Import: cURL / OpenAPI / Postman

Drag-and-drop or URL import. CodeLens for Express, Flask, FastAPI, C#.

LLM Evaluator · pip install qyrusai

0.94	0.89	0.91
Faithfulness	Relevance	Tool Quality

```
from qyrusai import AsyncQyrusAI

client = AsyncQyrusAI(api_key="...")

res = await client.evaluate_rag(request)

res['metrics']['faithfulness']
res['metrics']['tool_quality']
```

Qyrus customers see measurable impact — often within 90 days

73%
Reduction in Test Creation Time

4.2×
Faster Release Cycles

91%
AI Tests Pass on First Run

$2.1M
Avg Annual Savings per Enterprise

A challenger bank in UK

Deployed Qyrus across 14 teams. Eliminated 80% of manual regression. Shipped 3× faster in Q1.

Bank

From conversation to production-grade AI quality — in 3 moves

01

Book a Demo

See Qyrus live with your tech stack. We'll run a real test session on your app — not a canned walkthrough.

Schedule in 60 seconds →

02

Start a Free Trial

30 days, full platform access. Import your existing test suite or let our AI generate coverage from scratch.

No credit card needed →

03

Talk to an Expert

Our AI Quality Architects will map a deployment plan to your SDLC, compliance needs, and team structure.

Get your roadmap →

Summary of **Qyrus** – this is **who we are**

AI and **Test Automation** are **our DNA**

We test all things **digital/AI**

Try Qyrus AI

www.qyrus.com

QyrusAI qyrus qyrusAI

QYRUS

AI-Native Quality.
Built for the Age of Intelligent Software.

Thank you for your **time** today.

qyrus.com | Ravi@qyrus.com | Qapi.qyrus.com

The future of software quality is intelligent, adaptive, and already here.